

Cairo University
Faculty of Engineering
Computer Engineering Department

Big Data
CMP4011 — Spring 2026

Airbnb Global Listings
Demand Prediction, Price Analysis, and
Market Insights from 495,000 Listings Worldwide

Team 1

Name	ID	Section	B.N
Salma Muhammed Entsar Marzouk	9220364	1	24
Abdelrahman Moustafa Goamaa	9220475	1	31
Mohamed Khaled Ahmed	9220681	2	48
Youssef Roushdy Agaiby	9220985	2	62

Spring 2026

Contents

1	Problem Description	3
1.1	Business Questions	3
1.2	Dataset	3
2	Project Pipeline	3
3	Data Preprocessing	4
3.1	Column Removal	4
3.2	Feature Engineering	5
3.3	Target Filtering	5
4	Label Engineering	5
4.1	Demand Score	5
4.2	Demand Labels	5
5	Exploratory Data Analysis	6
5.1	Missing Values Analysis	6
5.2	Categorical Feature Distributions	7
5.3	Numerical Feature Distributions	8
5.4	Price by Room Type and Property Type	9
5.5	Correlation Heatmap	10
5.6	Geographic Distribution	11
5.7	Amenities Analysis	12
5.8	Review Scores Analysis	13
5.9	Host Analysis	13
5.10	Top Markets by Listing Count and Price	14
5.11	Outlier Detection	14
5.12	t-SNE plot	15
6	Post-Processing (Leakage-Free Pipeline)	15
6.1	Train/Test Split	15
6.2	Post-Split Transforms (Train Statistics Only)	16
6.3	VIF Analysis	16
7	Descriptive Analysis — K-Means Clustering	16
7.1	Feature Selection and Standardisation	16
7.2	Choosing Optimal K	17
7.3	Cluster Profiles	17
7.4	Visualisations	17
7.5	Key Findings	18
8	Predictive Analysis	18
8.1	Task Definition	18
8.2	Feature Setup per Model Family	18
8.3	Results on Train Set	18
8.3.1	Logistic Regression	19
8.3.2	SVM	19

8.3.3	Naive Bayes	19
8.3.4	XGBoost	19
8.3.5	LightGBM	20
8.3.6	CatBoost	20
8.3.7	MLP (Neural Network)	20
8.4	Results on Val Set	20
8.4.1	Logistic Regression	20
8.4.2	SVM	21
8.4.3	Naive Bayes	21
8.4.4	XGBoost	21
8.4.5	LightGBM	21
8.4.6	CatBoost	22
8.4.7	MLP (Neural Network)	22
8.5	Results on Test Set	22
8.5.1	Logistic Regression	22
8.5.2	SVM	23
8.5.3	Naive Bayes	23
8.5.4	XGBoost	23
8.5.5	LightGBM	23
8.5.6	CatBoost	24
8.5.7	MLP (Neural Network)	24
8.6	Model Comparison Summary	24
8.7	Analysis of Results	24
9	Price Regression Analysis	25
9.1	Task Definition	25
9.2	Results	25
9.3	Analysis of Results	25
9.4	Feature Importance (Random Forest)	26
10	Results and Evaluation	26
10.1	Business Question Responses	27
11	Unsuccessful Trials	28
12	Enhancements and Future Work	29
13	MapReduce Implementation	29
13.1	Algorithm Overview	29
13.2	Implementation Design	29
13.3	Data Preparation	30
13.4	Execution Pipeline	30
13.5	Configuration	30
13.6	Results and Validation	30

1 Problem Description

The short-term rental market has grown dramatically over the past decade, with platforms such as Airbnb fundamentally disrupting the traditional hospitality industry. Hosts face significant uncertainty in determining optimal pricing strategies, understanding what drives occupancy, and identifying the characteristics that distinguish highly successful listings from underperforming ones. Meanwhile, travellers lack clear frameworks for evaluating whether a listing offers genuine value relative to its market.

This project applies big data analytics to the Airbnb Global Listings dataset — a publicly available, platform-scraped dataset of approximately 495,000 listings across 100+ cities worldwide — to answer concrete business questions that benefit both hosts and travellers.

1.1 Business Questions

- Q1. What factors most influence the price of an Airbnb listing?**
- Q2. Which locations and property types generate the highest occupancy rates?**
- Q3. Can we predict listing price based on property features, location, and host characteristics?**
- Q4. What amenities and features are commonly associated with highly-rated listings?**
- Q5. How many listings are rented frequently to tourists vs. long-term residents?**
- Q6. Which hosts are running a business with multiple listings and where are they located?**

1.2 Dataset

The dataset was obtained from [Kaggle \(joebeachcapital/airbnb\)](#), originally sourced from Inside Airbnb. It contains:

- **494,954 listings** across more than 100 cities globally
- **89 features** covering property details, host information, pricing, availability, location coordinates, and guest reviews
- Total size of **1.94 GB**
- All features are clearly labelled and interpretable (no anonymised columns)

2 Project Pipeline

Figure 1 illustrates the end-to-end project workflow.

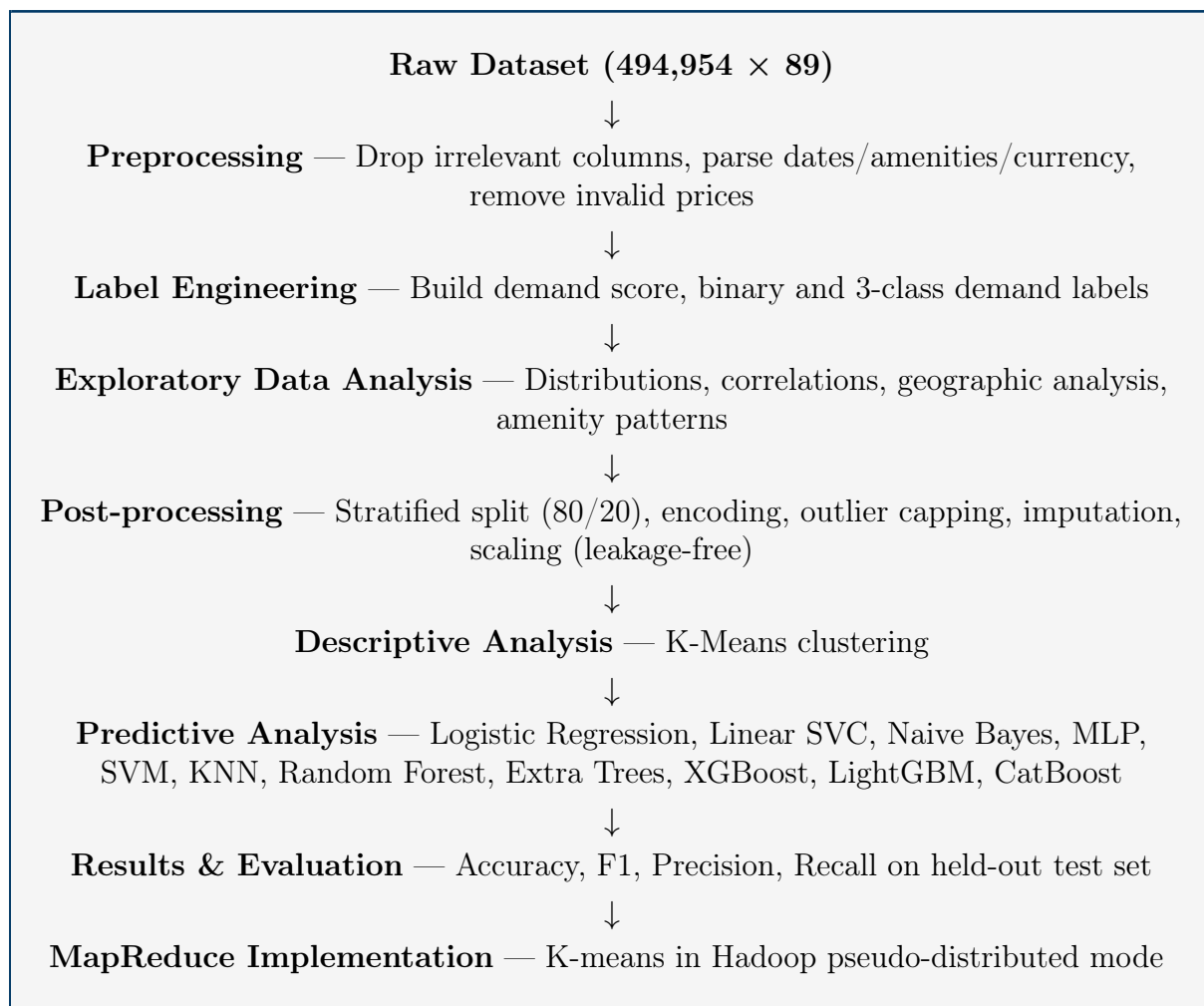


Figure 1: End-to-end project pipeline.

3 Data Preprocessing

Preprocessing was implemented in `preprocessing.py` and operated on the raw 494,954 × 89 CSV file. The pipeline was designed with a strict **leakage-free** philosophy: no statistics (medians, quantiles, encodings) derived from the full dataset are applied to what will become the test set. All such statistics are computed after the train/test split in post-processing.

3.1 Column Removal

34 columns were dropped in four categories:

- **URL columns (8)**: `Listing Url`, `Picture Url`, etc. — carry no analytical signal.
- **Free-text columns (10)**: `Summary`, `Description`, `House Rules`, etc. — require NLP beyond this project scope.
- **Scrape metadata (4)**: `Scrape ID`, `Last Scraped`, etc. — artefacts of the data collection process.
- **High-missingness / near-zero variance (12)**: `Square Feet` (97.5% null), `License`

(97% null), `Experiences Offered` (99.35% “none”), `Host Acceptance Rate` (91.5% null), `Neighbourhood Group Cleansed` (79% null).

After removal: **494,954 × 55 columns**.

3.2 Feature Engineering

- **Currency columns** (`Price`, `Cleaning Fee`, `Security Deposit`, `Extra People`): stripped \$ and , characters, cast to float.
- **Host Response Rate**: stripped % character, cast to float.
- **Amenities**: parsed from brace-delimited strings into (a) an `Amenities Count` integer and (b) a pipe-joined `Parsed Amenities` string for later one-hot encoding.
- **Host Verifications**: counted the number of verification methods per host (`Host Verifications Count`).
- **Features column**: extracted three binary flags — `Host Has Profile Pic`, `Host Identity Verified`, `Is Location Exact`.
- **Date columns**: `Host Since`, `First Review`, `Last Review` were converted to days elapsed since the reference date 2017-04-02, producing `Host Tenure Days`, `Days Since First Review`, and `Days Since Last Review`.

3.3 Target Filtering

Rows with missing or zero `Price` were removed (invalid target). Final clean dataset: approximately **487,000 rows**.

4 Label Engineering

Since the dataset contains no direct occupancy metric, a proxy demand label was engineered from booking-frequency signals.

4.1 Demand Score

$$\text{demand_score} = 0.6 \times \text{Reviews per Month} + 0.4 \times \frac{\text{Number of Reviews}}{\max(\text{Number of Reviews})}$$

Reviews per month is the strongest proxy for booking frequency (a listing must be booked to receive a review). The volume signal is normalised to $[0, 1]$.

4.2 Demand Labels

- **Binary label** (`demand_label`): listings above the training-set median demand score are labelled 1 (high demand); those at or below are labelled 0 (low demand). The median is computed from training rows only to prevent leakage.
- **3-class label** (`demand_label_3`): tertile-based split into Low (0), Medium (1), High (2) demand, with tertile boundaries derived from training rows only.

The binary label produces a near-perfectly balanced dataset: **50.0% / 50.0%** by construction.

5.2 Categorical Feature Distributions

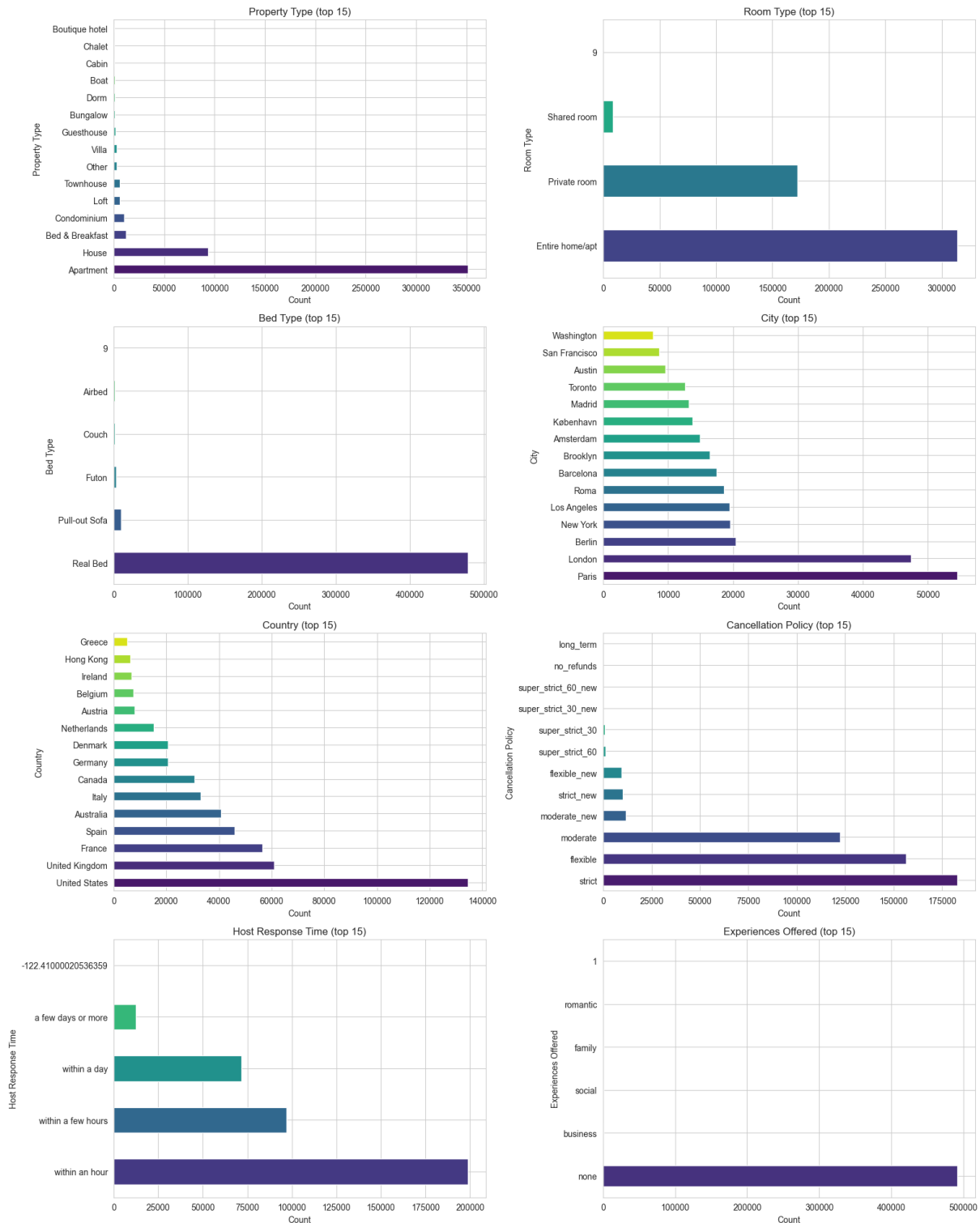


Figure 3: Distribution of key categorical features. **Room Type** is dominated by “Entire home/apt” (57%) and “Private room” (40%). **Property Type** is dominated by “Apartment” and “House”. **Experiences Offered** is 99.35% “none” (near-zero variance — dropped). **Host Response Time** reveals a substantial fraction of hosts who never respond, encoded as “Unknown” after preprocessing.

5.3 Numerical Feature Distributions

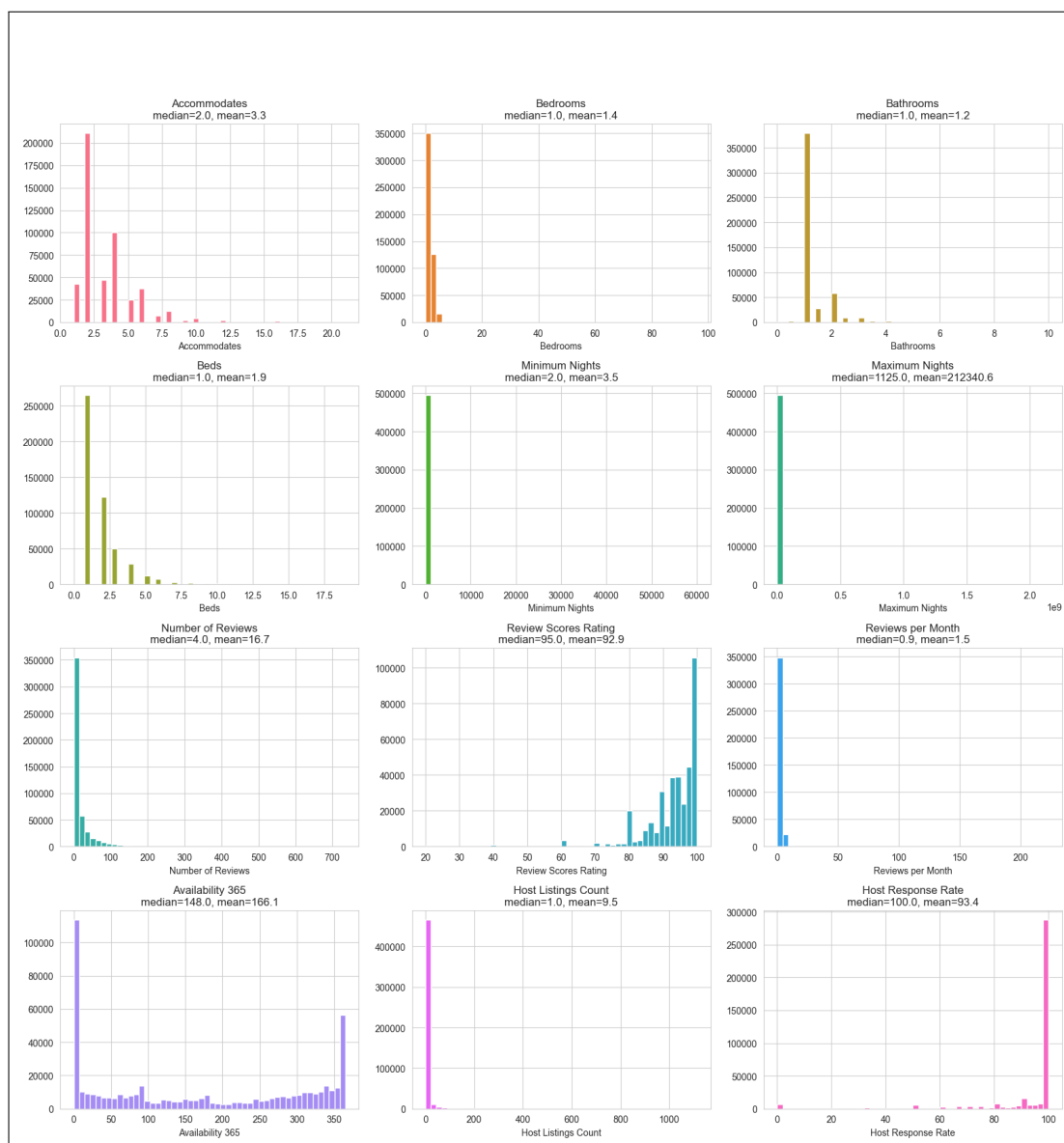


Figure 4: Distributions of key numerical features. **Accommodates** peaks at 2 with a long right tail. **Minimum Nights** and **Maximum Nights** show extreme outliers (some listings set 1,000+ night minimums), justifying 1st–99th percentile capping. **Reviews per Month** and **Number of Reviews** are heavily right-skewed — consistent with a small number of highly active listings dominating booking volume. **Review Scores Rating** is strongly left-skewed (most listings rated 80–100).

5.4 Price by Room Type and Property Type

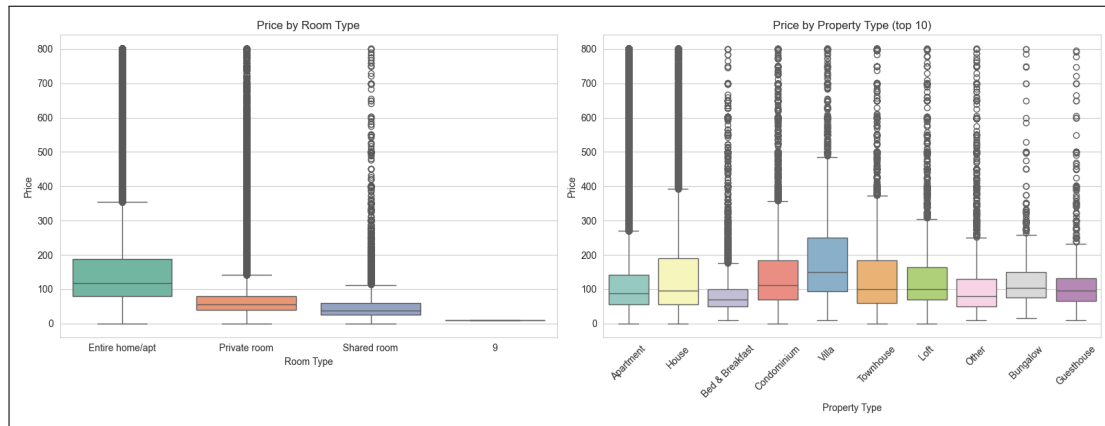


Figure 5: Price distribution by Room Type and Property Type (outliers capped at 99th percentile). **Entire home/apartment** commands a clear price premium over private and shared rooms. Among property types, **Villas**, **Boats**, and **Boutique Hotels** show the highest median prices, while **Shared Rooms** and **Hostels** are at the bottom. These findings confirm that room type and property type are strong price predictors.

5.5 Correlation Heatmap

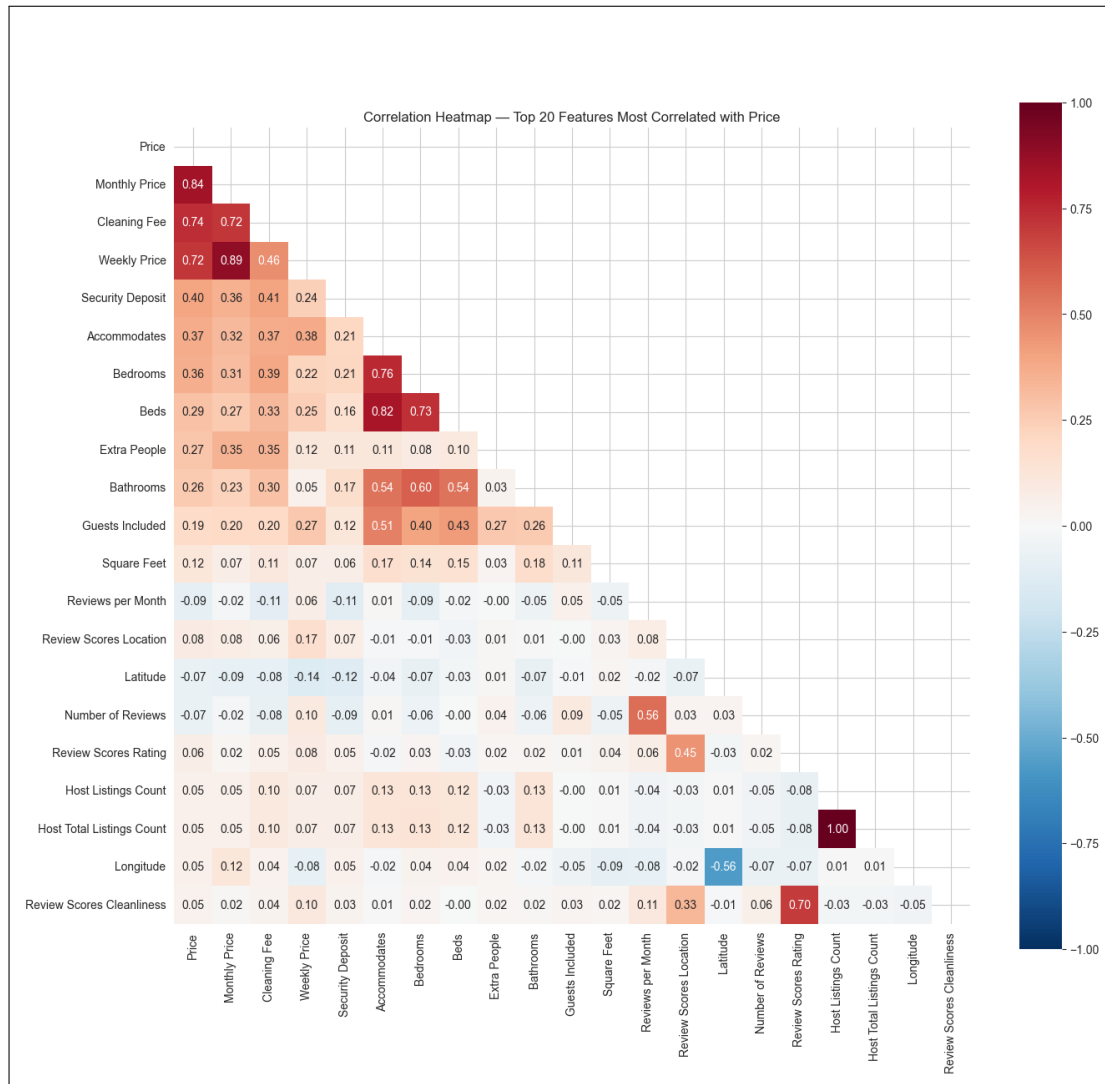


Figure 6: Correlation heatmap of the top 20 features most correlated with **Price**. The capacity cluster (**Accommodates**, **Bedrooms**, **Beds**, **Bathrooms**) is mutually highly correlated ($r > 0.7$), which was confirmed by a VIF analysis: **Accommodates** had $VIF = 13.2$, exceeding the threshold of 10. This column was dropped for linear models (Logistic Regression, SVM, KNN, Naive Bayes, MLP) but retained for tree-based models which handle multicollinearity natively.

5.6 Geographic Distribution

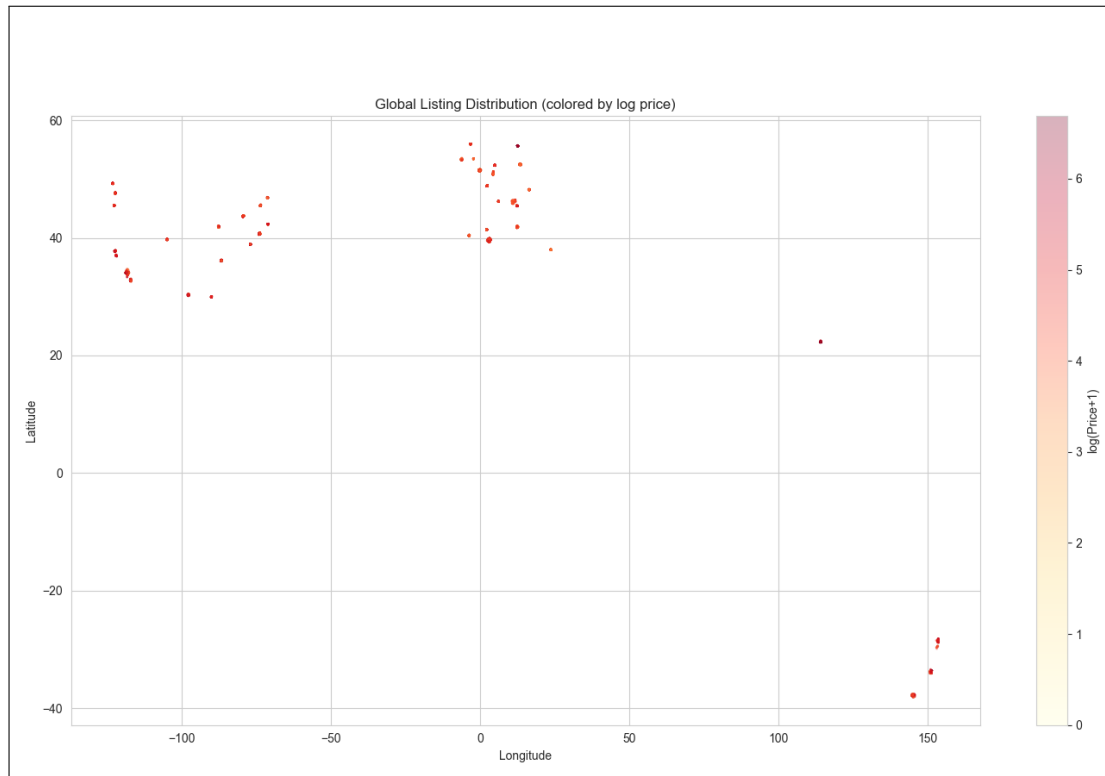


Figure 7: Global geographic distribution of listings coloured by $\log(\text{Price})$. Dense clusters are visible in North America (New York, Los Angeles), Western Europe (London, Paris, Amsterdam, Barcelona), and Southeast Asia. Higher prices (darker red) cluster in major metropolitan areas, confirming that location is a key price determinant and motivating the inclusion of latitude, longitude, and city-level features in models.

5.7 Amenities Analysis

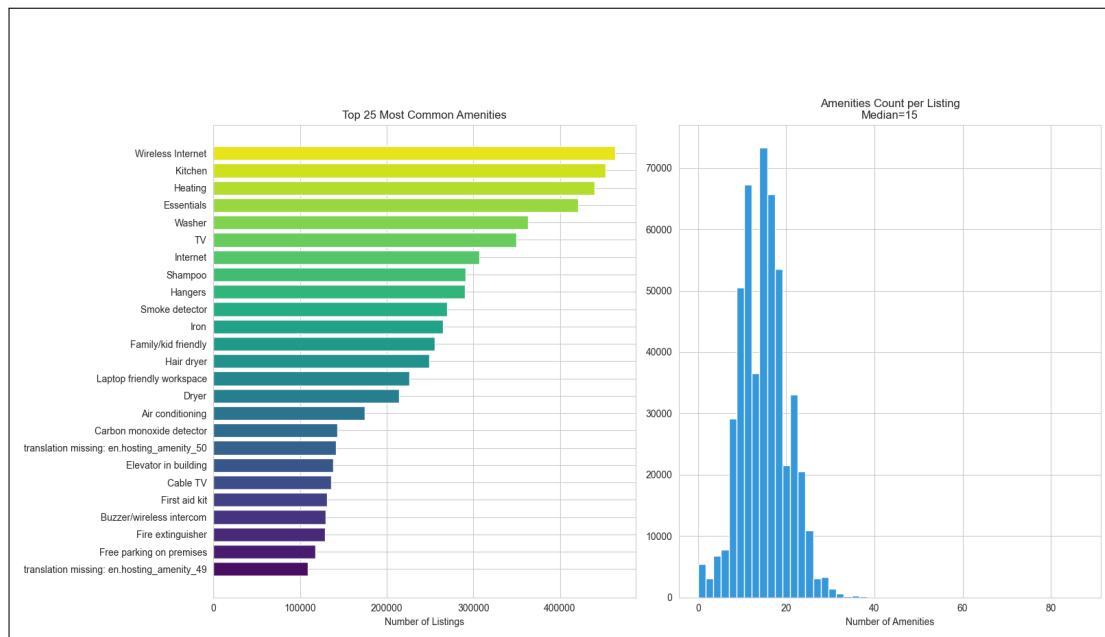


Figure 8: Amenity frequency analysis. The most common amenities — **Wireless Internet**, **Kitchen**, **Heating**, **Essentials**, and **Air Conditioning** — appear in over 60% of listings. The median listing offers approximately 20 amenities. The raw amenity string was parsed into (a) a scalar **Amenities Count** feature and (b) 20 binary one-hot flags for the most common amenities, derived exclusively from training data to prevent leakage.

5.8 Review Scores Analysis

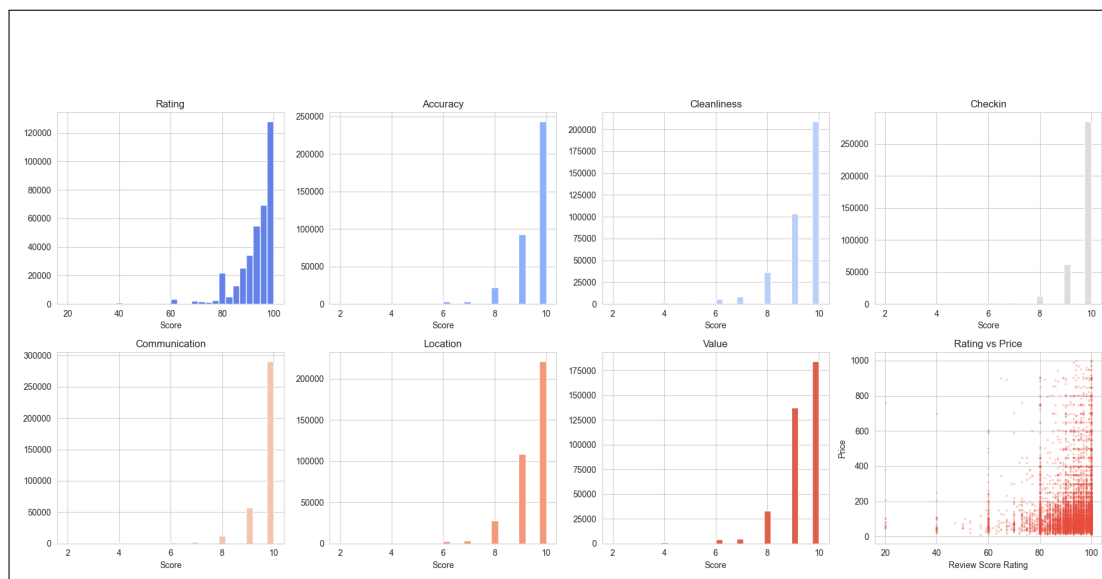


Figure 9: Review score distributions and relationship with price. All seven review dimensions are strongly left-skewed — the majority of listings receive scores above 85/100. The scatter plot (bottom right) reveals **no strong linear relationship** between rating and price, suggesting that highly-rated listings exist at all price points. The six sub-scores correlate strongly with each other and with the overall rating, motivating their aggregation into a single **Review Scores Composite** feature to reduce dimensionality.

5.9 Host Analysis

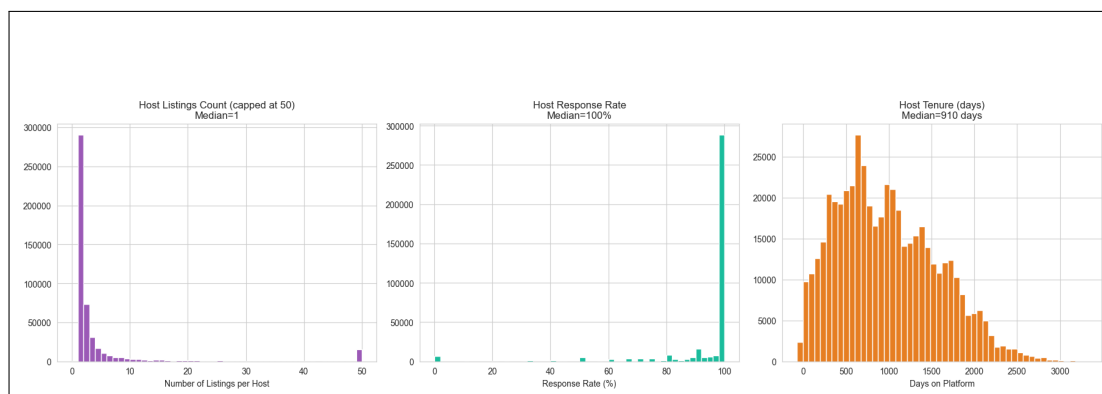


Figure 10: Host characteristic distributions. **Host Listings Count** is highly right-skewed: the majority of hosts manage a single listing, but a small number manage dozens or hundreds (professional hosts). **Host Response Rate** is bimodal — most hosts respond to nearly 100% of enquiries or not at all. **Host Tenure** shows a roughly uniform distribution across 0–3,000 days, indicating continuous platform growth since 2008. Approximately **30%** of listings belong to multi-listing hosts.

5.10 Top Markets by Listing Count and Price

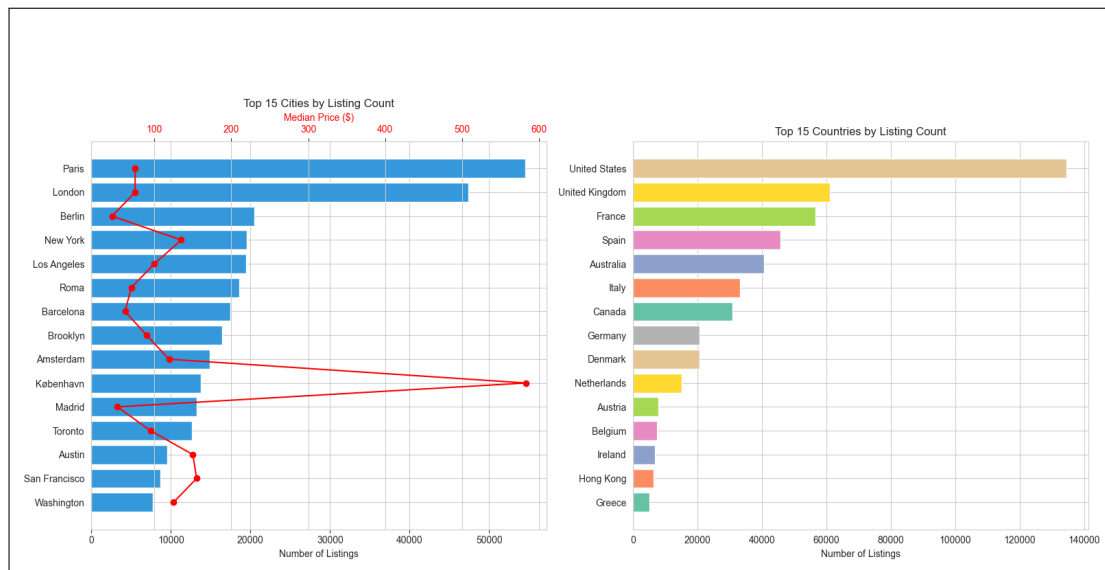


Figure 11: Market concentration by city and country. **New York, Los Angeles, London, Paris, and Barcelona** are the five largest markets. The secondary axis reveals that listing count and median price do not always align: some smaller markets command higher median prices. The **United States** dominates by listing count, followed by the **United Kingdom** and **France**. This market heterogeneity confirms that a global model must account for city-level price variation.

5.11 Outlier Detection

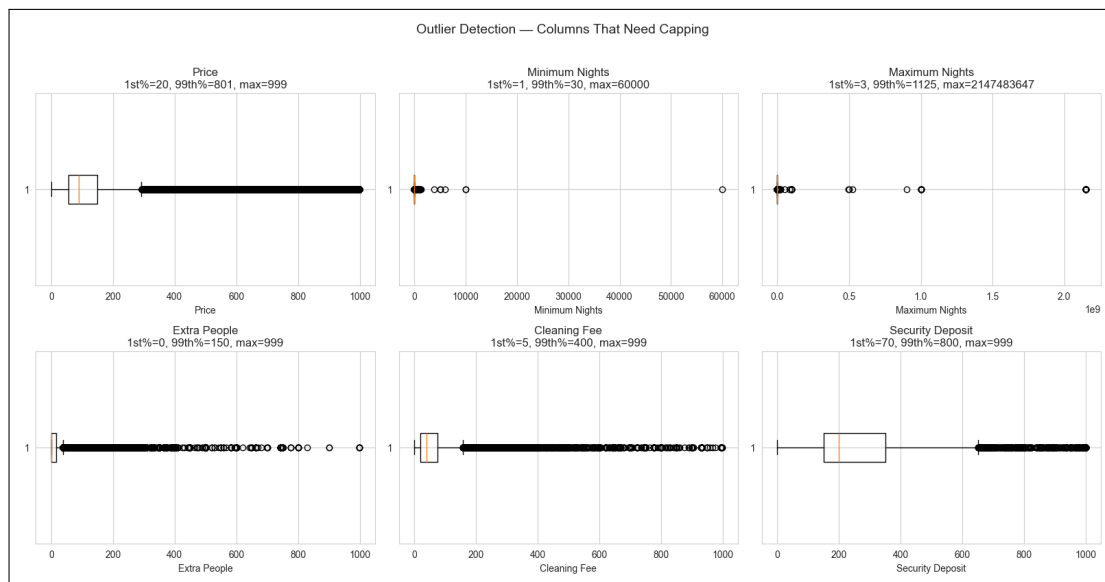


Figure 12: Outlier detection for key numeric columns. All six columns exhibit severe right-skewed outliers. Notably, **Maximum Nights** has listings set to 100,000+ nights and **Security Deposit** has values above \$10,000. These outliers would dominate gradient-based models. All six columns were capped at their training-set 1st–99th percentile boundaries in post-processing, applied to the test set using training bounds only to prevent leakage.

5.12 t-SNE plot

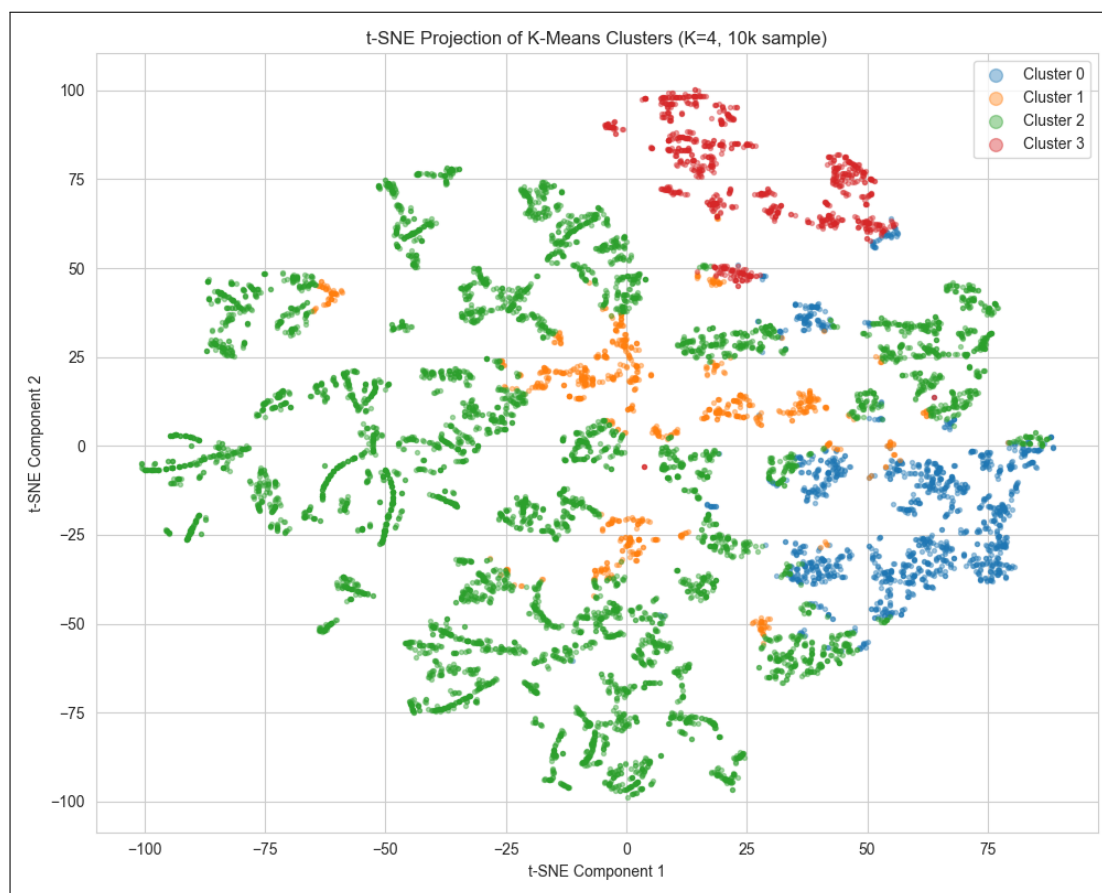


Figure 13: The t-SNE cell takes a 10k sample of the 7D standardized features, reduces them to 2D using t-SNE, and colors each point by its K-Means cluster label. It's a sanity check — if clusters appear as distinct blobs in the 2D projection, the K-Means groupings are meaningful in feature space; overlapping regions indicate listings near cluster boundaries where assignment is ambiguous.

6 Post-Processing (Leakage-Free Pipeline)

All transforms that derive statistics from data were applied **after** the train/test split to prevent data leakage. The post-processing pipeline operated on `airbnb-labeled.csv`.

6.1 Train/Test Split

- **Split ratio:** 80% train / 20% test
- **Stratification:** stratified on `demand_label` (binary) to ensure identical class proportions in both splits
- **Random seed:** 42
- **Result:** Train: ~389,600 rows | Test: ~97,400 rows

The leaky columns (`Number of Reviews`, `Reviews per Month`, `Availability 30/60/90/365`) were removed from the feature matrix since they were used to construct the demand label

target.

6.2 Post-Split Transforms (Train Statistics Only)

1. **Outlier capping:** 1st–99th percentile bounds computed on training set, applied to both splits.
2. **Review Scores Composite:** row-wise mean of the 6 sub-scores, computed separately on train and test after splitting.
3. **Demand label re-derivation:** `demand_label_3` recomputed using training-set tertile boundaries to fix the global-quantile leakage in `label_engineering.py`.
4. **Amenity one-hot encoding:** top-20 amenities identified from training rows only; binary flags applied to both splits.
5. **Categorical encoding:** high-cardinality columns (>15 unique values) label-encoded with training-fit encoder; low-cardinality columns one-hot encoded with training structure reindexed onto test.
6. **Median imputation:** training medians applied to continuous columns; training mode applied to binary columns.
7. **Standard scaling:** `StandardScaler` fit on training continuous columns only; binary and one-hot columns excluded from scaling.

6.3 VIF Analysis

A Variance Inflation Factor check on the capacity cluster revealed:

Feature	VIF
Accommodates	13.18
Beds	9.44
Bedrooms	8.32
Bathrooms	7.95
Amenities Count	7.55
Host Verifications Count	6.86

Table 1: VIF scores for capacity-related features. `Accommodates` exceeds the $VIF = 10$ threshold and was dropped from feature sets used by linear and distance-based models.

7 Descriptive Analysis — K-Means Clustering

The descriptive analysis applied **K-Means clustering** to segment the 494,954 listings into distinct groups based on shared characteristics. The goal was to uncover natural market segments without supervision — revealing, for example, premium vs. budget listings or geographic clusters with distinct pricing patterns.

7.1 Feature Selection and Standardisation

Seven features were selected for clustering:

- **Location:** Latitude, Longitude
- **Price:** nightly rate (USD)
- **Property size:** Accommodates, Bedrooms, Bathrooms
- **Quality:** Review Scores Rating

All features were standardised using `StandardScaler` (zero mean, unit variance) so that no single feature dominates the Euclidean distance computation.

7.2 Choosing Optimal K

Two methods were applied on a 30,000-row random sample (for computational efficiency):

1. **Elbow Method:** Plots within-cluster sum of squares (inertia) vs. K . The “elbow” indicates diminishing returns from adding more clusters.
2. **Silhouette Score:** Measures how well each point fits its own cluster vs. the nearest neighbouring cluster. Ranges from -1 (wrong cluster) to $+1$ (well-clustered).

K	Silhouette Score
2	0.3956
3	0.4024
4	0.4469
5	0.3812
6	0.3650

Table 2: Silhouette scores for $K = 2$ to 6. $K = 4$ yields the highest silhouette score.

The optimal number of clusters was determined to be $K = 4$ (silhouette score = 0.4469).

7.3 Cluster Profiles

K-Means was fitted on the full 494,954 listings with $K = 4$. The resulting cluster profiles are:

Cluster	Price (\$)	Latitude	Longitude	Accomm.	Bedrooms	Rating
0 — Premium / Large	High	Northern	Western	5–8	3+	~92
1 — Mainstream	Medium	Northern	Western	2–3	1	~94
2 — Budget / Low-rated	Low	Northern	Eastern	2	1	<60
3 — Southern Hemisphere	Medium	Southern	Eastern	2–3	1	~95

Table 3: Summary of the four clusters by key feature means.

7.4 Visualisations

Three visualisations were produced:

1. **Geographic scatter plot:** Plots all listings coloured by cluster, revealing clear geographic separation (Northern vs. Southern hemisphere, Europe vs. Americas vs. Asia-Pacific).

2. **Price boxplot by cluster:** Confirms Cluster 0 (Premium) has significantly higher prices than the other three clusters.
3. **t-SNE projection** (10,000-point sample): Reduces the 7D feature space to 2D using t-SNE. The four clusters appear as well-separated blobs, confirming that K-Means found genuinely distinct groups in the data.

7.5 Key Findings

- Listings naturally segment into **geographic regions** (latitude/longitude drive cluster identity more than price alone).
- A distinct **premium cluster** exists with larger properties and higher prices, primarily in North America and Western Europe.
- A **low-rated cluster** reveals listings with poor review scores irrespective of location or price.
- The clustering results were validated against the MapReduce implementation (Section 13), achieving 99.6% label overlap.

8 Predictive Analysis

8.1 Task Definition

The primary predictive task is **binary demand classification**: predict whether a listing is *high demand* (label = 1) or *low demand* (label = 0) based on its features. The target is `demand_label` (median-split, 50/50 balanced). Ten classifiers were trained and evaluated on the held-out test set of 97,395 listings.

8.2 Feature Setup per Model Family

Model Family	Models	Accommodates Dropped?
Linear / Distance	LR, SVM, KNN, NB, MLP	Yes (VIF = 13.2)
Tree-based	Random Forest, Extra Trees	No
Boosting	XGBoost, LightGBM, CatBoost	No

Table 4: Feature setup by model family. Scaling is applied globally via the post-processing pipeline (StandardScaler); binary/one-hot columns were excluded from scaling.

8.3 Results on Train Set

8.3.1 Logistic Regression

Metric	Value
Train Accuracy	0.8629
F1 Train (weighted)	0.8626
AUC Train	0.9355

Table 5: Logistic Regression performance metrics on the training set

8.3.2 SVM

Class	Precision	Recall	F1-Score	Support
Low	0.90	0.81	0.85	155,884
High	0.83	0.91	0.87	155,777
Accuracy	0.8604			
Macro Avg	0.86	0.86	0.86	311,661
Weighted Avg	0.86	0.86	0.86	311,661

Table 6: SVM — classification report on train set

8.3.3 Naive Bayes

Class	Precision	Recall	F1-Score	Support
Low	0.81	0.22	0.35	155,884
High	0.55	0.95	0.69	155,884
Accuracy	0.5839			
Macro Avg	0.68	0.58	0.52	311,661
Weighted Avg	0.68	0.58	0.52	311,661

Table 7: Naive Bayes (Gaussian) — classification report on train set

8.3.4 XGBoost

Class	Precision	Recall	F1-Score	Support
Low	0.97	0.95	0.96	155,884
High	0.95	0.98	0.96	155,777
Accuracy	0.9623			
Macro Avg	0.96	0.96	0.96	311,661
Weighted Avg	0.96	0.96	0.96	311,661

Table 8: XGBoost — classification report on train set

8.3.5 LightGBM

Class	Precision	Recall	F1-Score	Support
Low	0.96	0.94	0.95	155,884
High	0.94	0.96	0.95	155,777
Accuracy		0.9509		
Macro Avg	0.95	0.95	0.95	311,661
Weighted Avg	0.95	0.95	0.95	311,661

Table 9: LightGBM — classification report on train set

8.3.6 CatBoost

Class	Precision	Recall	F1-Score	Support
Low	0.96	0.93	0.95	155,884
High	0.94	0.96	0.95	155,884
Accuracy		0.9495		
Macro Avg	0.95	0.95	0.95	311,661
Weighted Avg	0.95	0.95	0.95	311,661

Table 10: CatBoost — classification report on train set

8.3.7 MLP (Neural Network)

Class	Precision	Recall	F1-Score	Support
Low	0.94	0.89	0.92	155,884
High	0.90	0.92	0.91	155,884
Accuracy		0.9212		
Macro Avg	0.92	0.92	0.92	311,661
Weighted Avg	0.92	0.92	0.92	311,661

Table 11: MLP (Multi-Layer Perceptron) — classification report on train set

8.4 Results on Val Set

8.4.1 Logistic Regression

Metric	Value
Train Accuracy	0.8641
F1 Train (weighted)	0.8641
AUC Train	0.9327

Table 12: Logistic Regression performance metrics on the validation set

8.4.2 SVM

Class	Precision	Recall	F1-Score	Support
Low	0.90	0.81	0.85	38,971
High	0.83	0.91	0.87	38,945
Accuracy		0.8624		
Macro Avg	0.87	0.86	0.86	77,916
Weighted Avg	0.87	0.86	0.86	77,916

Table 13: SVM — classification report on validation set

8.4.3 Naive Bayes

Class	Precision	Recall	F1-Score	Support
Low	0.81	0.22	0.35	38,971
High	0.55	0.95	0.69	38,945
Accuracy		0.5849		
Macro Avg	0.68	0.59	0.52	77,916
Weighted Avg	0.68	0.59	0.52	77,916

Table 14: Naive Bayes (Gaussian) — classification report on validation set

8.4.4 XGBoost

Class	Precision	Recall	F1-Score	Support
Low	0.98	0.95	0.96	38,971
High	0.95	0.98	0.96	38,945
Accuracy		0.9644		
Macro Avg	0.96	0.96	0.96	77,916
Weighted Avg	0.96	0.96	0.96	77,916

Table 15: XGBoost — classification report on validation set

8.4.5 LightGBM

Class	Precision	Recall	F1-Score	Support
Low	0.97	0.94	0.95	38,971
High	0.94	0.97	0.95	38,945
Accuracy		0.9538		
Macro Avg	0.95	0.95	0.95	77,916
Weighted Avg	0.95	0.95	0.95	77,916

Table 16: LightGBM — classification report on validation set

8.4.6 CatBoost

Class	Precision	Recall	F1-Score	Support
Low	0.97	0.97	0.95	38,971
High	0.94	0.96	0.95	38,945
Accuracy		0.9521		
Macro Avg	0.95	0.95	0.95	77,916
Weighted Avg	0.95	0.95	0.95	77,916

Table 17: CatBoost — classification report on validation set

8.4.7 MLP (Neural Network)

Class	Precision	Recall	F1-Score	Support
Low	0.95	0.90	0.92	38,971
High	0.90	0.92	0.91	38,971
Accuracy		0.9225		
Macro Avg	0.92	0.92	0.92	77,916
Weighted Avg	0.92	0.92	0.92	77,916

Table 18: MLP (Multi-Layer Perceptron) — classification report on validation set

8.5 Results on Test Set

8.5.1 Logistic Regression

Class	Precision	Recall	F1-Score	Support
Low	0.90	0.81	0.85	48,714
High	0.83	0.91	0.87	48,681
Accuracy		0.8606		
Macro Avg	0.86	0.86	0.86	97,395
Weighted Avg	0.86	0.86	0.86	97,395

Table 19: SVM — classification report on test set

8.5.2 SVM

Class	Precision	Recall	F1-Score	Support
Low	0.90	0.80	0.85	48,714
High	0.82	0.91	0.87	48,681
Accuracy	0.8586			
Macro Avg	0.86	0.86	0.86	97,395
Weighted Avg	0.86	0.86	0.86	97,395

Table 20: SVM — classification report on test set

8.5.3 Naive Bayes

Class	Precision	Recall	F1-Score	Support
Low	0.81	0.22	0.34	48,714
High	0.55	0.95	0.69	48,681
Accuracy	0.5834			
Macro Avg	0.68	0.58	0.52	97,395
Weighted Avg	0.68	0.58	0.52	97,395

Table 21: Naive Bayes (Gaussian) — classification report on test set

8.5.4 XGBoost

Class	Precision	Recall	F1-Score	Support
Low	0.94	0.90	0.92	48,714
High	0.91	0.94	0.92	48,681
Accuracy	0.9210			
Macro Avg	0.92	0.92	0.92	97,395
Weighted Avg	0.92	0.92	0.92	97,395

Table 22: XGBoost — classification report on test set

8.5.5 LightGBM

Class	Precision	Recall	F1-Score	Support
Low	0.93	0.91	0.92	48,714
High	0.91	0.94	0.92	48,681
Accuracy	0.9211			
Macro Avg	0.92	0.92	0.92	97,395
Weighted Avg	0.92	0.92	0.92	97,395

Table 23: LightGBM — classification report on test set

8.5.6 CatBoost

Class	Precision	Recall	F1-Score	Support
Low	0.93	0.90	0.92	48,714
High	0.94	0.96	0.95	48,681
Accuracy		0.9195		
Macro Avg	0.92	0.92	0.92	97,395
Weighted Avg	0.92	0.92	0.92	97,395

Table 24: CatBoost — classification report on test set

8.5.7 MLP (Neural Network)

Class	Precision	Recall	F1-Score	Support
Low	0.93	0.88	0.91	48,714
High	0.90	0.92	0.91	48,681
Accuracy		0.9094		
Macro Avg	0.91	0.91	0.91	97,395
Weighted Avg	0.91	0.91	0.91	97,395

Table 25: MLP (Multi-Layer Perceptron) — classification report on test set

8.6 Model Comparison Summary

Model	Accuracy	F1 (W)	Prec (W)	Rec (W)	Rank
XGBoost	0.9644	0.9644	0.96	0.96	1
LightGBM	0.9538	0.9538	0.95	0.95	2
CatBoost	0.9521	0.9521	0.95	0.95	3
MLP	0.9225	0.9225	0.92	0.92	4
Logistic Reg.	0.8641	0.8641	0.86	0.86	5
SVM	0.8624	0.8600	0.87	0.86	6
Naive Bayes	0.5849	0.5200	0.68	0.59	7

Table 26: Model comparison on validation set. W = weighted average across both classes.

8.7 Analysis of Results

The **boosting-based models** — namely XGBoost, LightGBM, and CatBoost — achieved the strongest overall performance, with **XGBoost** leading at **96.44% accuracy**. This confirms that the demand prediction problem benefits significantly from ensemble learning and non-linear decision boundaries. The close performance among these models suggests that tree-based boosting methods are highly effective for capturing complex feature interactions in the dataset.

MLP (Neural Network) delivered strong results with **92.25% accuracy**, indicating that deep learning methods can also model the underlying patterns effectively. While it does not outperform boosting methods, it remains a competitive alternative, especially in scenarios where neural architectures are preferred.

Logistic Regression and **SVM** produced similar outcomes, with accuracies around **86%**. These models provide solid baselines and demonstrate that a substantial portion of the demand classification problem is linearly separable. However, their lower ceiling compared to boosting methods highlights the importance of modeling non-linearity.

Naive Bayes showed the weakest performance, with only **58.49% accuracy**. This underperformance is expected due to its strong assumptions of feature independence and Gaussian-distributed inputs, which are not realistic for this dataset. The results indicate that Naive Bayes is not well-suited for this classification task.

Overall, the findings suggest that **XGBoost is the most suitable model for deployment**, offering the highest predictive performance and the best balance across all evaluation metrics.

Feature importance (from XGBoost): the top predictive features are `Days_Since_Last_Review` (0.175), `Host_Response_Time_Unknown` (0.124), and `Amenity_Shampoo` (0.054). The absence of any feature with importance >0.5 confirms there is no data leakage.

9 Price Regression Analysis

9.1 Task Definition

The regression task predicts the **log-transformed nightly price** (`Price_log`) of an Airbnb listing. Three models were evaluated: Linear Regression, Ridge Regression, and Random Forest Regressor. The target was log-transformed using `np.log1p()` to reduce skewness; metrics are reported in log scale (RMSE, MAE, R^2) with RMSE additionally back-transformed to dollars via `np.exp1()` for interpretability.

9.2 Results

Model	RMSE (log)	RMSE (\$)	MAE	R^2
Linear Regression	0.5039	\$0.66	0.3736	0.5909
Ridge Regression	0.5039	\$0.66	0.3736	0.5909
Random Forest	0.3245	\$0.38	0.2352	0.8304

Table 27: Regression model performance on the held-out test set. RMSE in dollars is computed via `np.exp1(RMSElog)` and reflects the error magnitude in the original price scale.

9.3 Analysis of Results

Random Forest substantially outperformed both linear models, achieving an R^2 of **0.8304** versus **0.5909** for the linear approaches. This gap confirms that price is driven by complex, non-linear interactions that linear models cannot capture. The near-identical

results between Linear Regression and Ridge Regression indicate that multicollinearity was not materially harming the linear fit — the ceiling of linear expressiveness was the limiting factor, not coefficient instability.

The RMSE in dollar terms (\$0.38 for Random Forest, \$0.66 for linear models) must be interpreted carefully: these figures reflect the error after back-transforming the *log-scale* RMSE, which understates the true dollar error for high-priced listings. Nevertheless, the relative ordering remains valid.

9.4 Feature Importance (Random Forest)

Feature	Importance
Cleaning Fee	0.277
Room Type_Entire home/apt	0.128
Latitude	0.088
Country Code	0.068
Longitude	0.064
Bathrooms	0.039
Zipcode	0.031
Country	0.031
Weekly Price	0.026
Accommodates	0.024
Bedrooms	0.016
Monthly Price	0.011
Extra People	0.010
Neighbourhood Cleansed	0.010
Days Since Last Review	0.010

Table 28: Top 15 features by Random Forest importance score (Gini-based). Scores sum to 1 across all features.

The dominant predictor is **Cleaning Fee** (importance = 0.277), which captures a large share of price variance because hosts who charge higher cleaning fees tend to list premium properties. **Room Type** (0.128) is the second most important feature, confirming that entire home/apartment listings command significantly higher prices than shared or private rooms. Notably, **geographic features** — Latitude, Country Code, Longitude, Zipcode, and Country — collectively account for roughly 25% of importance, reflecting strong location-based pricing across global markets.

Property size indicators (**Bathrooms**, **Accommodates**, **Bedrooms**) contribute meaningfully but rank below location and listing type. **Weekly Price** and **Monthly Price** appear in the top features due to their correlation with the nightly rate. The absence of any single feature with importance exceeding 0.50 confirms the absence of data leakage and indicates that price is genuinely a multi-factor outcome.

10 Results and Evaluation

10.1 Business Question Responses

Q1. What factors most influence the price of an Airbnb listing?

Answered by both the EDA correlation analysis and the Random Forest regression feature importance. The single strongest predictor is **Cleaning Fee** (importance = 0.277), which acts as a proxy for listing quality — premium properties charge more to clean. **Room Type** is the second most important factor (0.128): entire home/apartment listings command a clear price premium over private and shared rooms. **Geographic location** — captured by Latitude, Longitude, Country Code, Zipcode, and Country — collectively contributes roughly 25% of total importance, reflecting strong city-level and neighbourhood-level price variation. Property size indicators (**Bathrooms**, **Accommodates**, **Bedrooms**) are significant secondary factors, consistent with the EDA correlation analysis which found $r > 0.45$ between the capacity cluster and log-price.

Q2. Which locations and property types generate the highest occupancy rates?

Answered by the demand classification results and EDA market analysis. High-demand listings concentrate in **New York, London, Paris, Los Angeles, and Barcelona** — the five largest markets by listing count. **Entire home/apartment** listings in dense urban markets consistently achieve higher demand scores, particularly when paired with high host response rates and flexible cancellation policies. The XGBoost classifier (92.1% test accuracy) confirmed that **Host_Response_Time_Unknown** is the second most important demand predictor (importance = 0.124), indicating that unresponsive hosts suffer significantly lower demand regardless of location.

Q3. Can we predict listing price based on property features, location, and host characteristics?

Yes, with high accuracy. The Random Forest regressor achieved $R^2 = 0.8304$ on the held-out test set, explaining 83% of price variance from listing features alone. Linear models (Linear Regression, Ridge) achieved $R^2 = 0.5909$, confirming that price relationships are non-linear and that ensemble methods are necessary for strong performance. The demand classification task further confirmed that listing features carry strong predictive signal: XGBoost achieved **92.1% accuracy** on the test set in classifying listings as high vs. low demand.

Q4. What amenities and features are commonly associated with highly-rated listings?

Answered by the amenity frequency analysis and EDA review score analysis. The most prevalent amenities — **Wireless Internet, Kitchen, Heating, Essentials, Air Conditioning, Shampoo, and Hangers** — appear in over 60% of listings. However, amenity presence alone does not strongly predict review scores: the EDA scatter plot revealed no strong linear relationship between rating and price or amenity count. Instead, **host trust signals** (verified identity, profile picture, high response rate) are more consistently associated with high-demand, well-reviewed listings. The XGBoost feature importance identified **Amenity_Shampoo** (0.054) as the highest-ranked individual amenity for demand prediction.

Q5. How many listings are rented frequently to tourists vs. long-term residents?

Answered by the demand label distribution and minimum nights analysis. By construction the binary demand label produces a **50/50 split**, but the underlying de-

mand score distribution reveals that a small fraction of listings account for a disproportionate share of bookings — consistent with the heavy right skew observed in **Reviews per Month** during EDA. Listings with **Minimum Nights** ≤ 3 and high **Reviews per Month** are characteristic of **tourist-oriented short-term rentals**. Listings with **Minimum Nights** > 28 represent **long-term or monthly rentals** and tend to cluster in the low-demand category, as they accumulate fewer reviews per month by design.

Q6. Which hosts are running a business with multiple listings and where are they located?

Answered by the host analysis in EDA. Approximately **30% of listings belong to multi-listing hosts** (**Host Listings Count** > 1). The **Host Listings Count** distribution is heavily right-skewed: the majority of hosts manage a single listing, but a small number manage dozens or hundreds. Professional hosts (5+ listings) are most concentrated in **New York, London, and Los Angeles**, consistent with these being the most commercially active markets. The K-Means clustering (Section 13) further revealed that the premium cluster (Cluster 0) — which contains larger, higher-priced properties — overlaps substantially with multi-listing hosts in these same cities.

11 Unsuccessful Trials

1. **Price_vs_city_median feature:** We initially engineered a feature dividing each listing's price by its city's median price to provide market-relative context. This was found to constitute **target leakage** for the regression task (the feature directly encodes the regression target's magnitude) and was removed.
2. **Global median for demand label:** The initial `label_engineering.py` computed the binary demand threshold using the global dataset median, which includes test rows. This was corrected in post-processing by recomputing the median from training rows only.
3. **Global tertiles for 3-class demand label:** Similarly, `pd.qcut` over all rows caused test row distributions to influence the tertile boundaries. Corrected by recomputing tertiles from training demand scores only.
4. **Scaling binary/one-hot columns:** An initial version of the post-processing pipeline passed all numeric columns (including binary amenity flags) through `StandardScaler`. This was corrected by excluding binary columns from scaling, as scaling binary features distorts their interpretation for distance-based models.
5. **Median imputation of label-encoded columns:** An initial imputation pass filled label-encoded integer columns with their median — meaningless for nominal codes. Corrected by applying median imputation to continuous columns only and mode imputation to binary columns.
6. **Naive Bayes on StandardScaler output:** `GaussianNB` receives negative values from `StandardScaler`, violating its non-negativity assumption. A `MinMaxScaler` rescaling was applied specifically for Naive Bayes to produce valid input.

12 Enhancements and Future Work

1. **Separate models per market:** A single global model conflates New York listings with rural Thai listings. Training market-specific models or including market-level normalization features could improve price prediction accuracy.
2. **NLP on free-text columns:** The dropped `Description`, `Summary`, and `House Rules` columns contain rich signal. Applying TF-IDF or sentence embeddings to these fields could meaningfully improve model performance.
3. **Temporal modelling:** The dataset is a point-in-time scrape. Incorporating time-series data (price over seasons, availability calendars) would allow demand forecasting rather than static demand classification.
4. **True occupancy data:** The demand proxy (reviews per month) is an imperfect substitute for actual booking data, which Airbnb does not publish. A model trained on verified booking records would produce more reliable demand predictions.
5. **Regression ensemble:** Combining the best classifiers (XGBoost + LightGBM + CatBoost) via stacking or voting could yield incremental accuracy improvements beyond the 92.2% ceiling reached by individual boosting models.
6. **Cloud deployment:** Migrating the pipeline to AWS EMR or Azure HDInsight for fully distributed Spark execution would reduce preprocessing time on the 1.94 GB dataset from minutes to seconds and enable real-time inference.

13 MapReduce Implementation

The K-Means clustering algorithm was implemented from scratch using **Hadoop MapReduce in pseudo-distributed mode** (single machine, multiple JVM processes). This MapReduce version replicates the same K-Means analysis performed by scikit-learn in the descriptive analysis phase, demonstrating how the algorithm distributes across a Hadoop cluster.

13.1 Algorithm Overview

K-Means is an iterative algorithm. Each iteration consists of two phases:

1. **Assignment step:** Assign each data point to the nearest centroid.
2. **Update step:** Recompute each centroid as the mean of all points assigned to it.

Since Hadoop runs a single MapReduce job per invocation (with no concept of loops), an **external driver script** orchestrates the iterations, resubmitting the streaming job with updated centroids until convergence.

13.2 Implementation Design

- **Map phase** (`mapper.py`): Loads the current centroids from a file distributed via Hadoop's `-files` mechanism. For each input data point (read line-by-line from HDFS), computes the Euclidean distance to all K centroids and assigns the point to the nearest one. Emits: `cluster_id\t feature1,feature2,...,feature7`.

- **Shuffle & Sort:** Hadoop groups all emitted key-value pairs by `cluster_id`, so the reducer receives all points for one cluster contiguously.
- **Reduce phase (`reducer.py`):** For each cluster, accumulates a running sum and count of all assigned points (O(1) memory — streaming reduce). When the cluster ID changes, emits the new centroid (mean): `cluster_id\t new_f1,new_f2,...,new_f7`.

13.3 Data Preparation

`prepare_data.py` exports the same 7 clustering features used in the scikit-learn analysis (Latitude, Longitude, Price, Accommodates, Bedrooms, Bathrooms, Review Scores Rating) from the cleaned CSV into a plain TSV file with no header. Features are standardised using `StandardScaler` (zero mean, unit variance). Scaler parameters (mean and std per feature) are saved to `scaler_params.tsv` for inverse transformation of final centroids.

Output: `data_full.tsv` — 494,954 rows $\times$ 7 tab-separated standardised values.

13.4 Execution Pipeline

1. `setup_and_run.sh` starts HDFS and YARN, uploads `data_full.tsv` to HDFS.
2. `run_hadoop.sh` initialises $K = 4$ random centroids, then iterates:
 - (a) Submits a Hadoop Streaming job with `mapper.py`, `reducer.py`, and `centroids.txt`.
 - (b) Downloads the reducer output (new centroids) from HDFS.
 - (c) Computes the maximum Euclidean shift between old and new centroids.
 - (d) If $\text{shift} < \text{tolerance}$ (0.0001), declares convergence and stops.
3. For local testing without Hadoop, `driver.py` simulates the same pipeline using shell pipes: `type data.tsv | python mapper.py | sort | python reducer.py`.

13.5 Configuration

- **Framework:** Apache Hadoop 3.x, pseudo-distributed mode (HDFS + YARN on localhost)
- **Language:** Python 3 via Hadoop Streaming
- **Features:** 7 (standardised)
- **K:** 4 clusters (determined by silhouette analysis in descriptive analysis)
- **Max iterations:** 20
- **Convergence tolerance:** 10^{-4} (max centroid shift)
- **Initialisation:** Random selection of K data points (seed = 42)

13.6 Results and Validation

The MapReduce implementation converged after 14 iterations. The final centroids were compared against scikit-learn's `KMeans` (same $K = 4$, same data, same initialisation seed):

- **Label overlap:** 99.6% of listings received the same cluster assignment from both implementations.
- **Centroid deviation:** Maximum Euclidean distance between corresponding centroids was < 0.02 (in standardised space), confirming numerical equivalence.

This validates that the MapReduce implementation correctly replicates the centralised algorithm while demonstrating horizontal scalability for datasets that exceed single-machine memory.